Arduino Modbus Test

Purpose

This test establishes a framework for Modbus communications on the Arduino Opto which can be used for future integration. Holding registers and input registers are set up to demonstrate the ability to read telemetry data and to send commands. A full cycle of command and response is tested, enabling us to verify that the command was received properly, and a heartbeat is also established to verify that the communication channel remains open.

Description

In this test, the Arduino Opto acts as a Modbus client, while a Modbus server is running on the testing laptop. This server is set up using OpenModSim. The laptop will run at IP address 172.16.0.1 and the Arduino will run at IP address 172.16.0.2

The Arduino will be set up to run with two input registers. The first will send a device ID with a value of 1234. The second will send a heartbeat counter updating from 1 to 10, incrementing once per second and resetting from 10 to 1 when it finishes. These registers will be used to test telemetry reporting and heartbeat functionality. The device ID will be at input register 100, and the heartbeat will be at input register 101.

There will be three holding registers. The command will be sent from the laptop to the Arduino on holding register 100. The Arduino will then reflect that command on holding register 101. The Arduino will then multiply the command by 2 and send that value out on holding register 102. This will establish that a command can be sent and verified by reflecting it, and that the Arduino is able to take an action based on that command and send a result back.

The test consists of first reading the device ID to confirm that telemetry is being received. Then the heartbeat is read at least twice to confirm that it is updating by verifying that it shows two different values at two different times. Next, a command value of 10 is sent from the laptop. The reflected value will then be read, expecting a value of 10. The doubled value will then be read, expecting a value of 20. Finally, a command value of 3 is sent from the laptop. The reflected value will be read, expecting a value of 3. The doubled value is read, expecting a value of 6.

Procedure

	Step	Description	Result
0	Initial Conditions	The Arduino and the laptop are connected	

		to the same subnet using ethernet cables. The Modbus connection between the Arduino and the laptop has been established. Attach a screenshot from ModbusPal in the Result column to show that the connection is established.	
1	Read telemetry	Use OpenModSim to read input register 100. The expected value is 1234. Attach a screenshot from ModbusPal in the Result column to show that the expected value was received.	
2	Read heartbeat counter	Use OpenModSim to read input register 101. The expected value is between 1 and 10. Attach a screenshot from ModbusPal in the Result column to show that the expected value was received.	
3	Confirm heartbeat counter is incrementing	Use OpenModSim to read input register 101 again. The expected value is between 1 and 10, and it should be a different value than that read in step 2. Attach a screenshot from ModbusPal in the Result column to show that the expected value was received.	
4	Send command of 10	Use OpenModSim to send a value of 10 to holding register 100. Read holding registers 101 and 102. The expected value of holding register 101 is 10. The expected value of holding register 102 is 20. Attach a screenshot from ModbusPal in the Result column to show that the expected value was received.	
5	Send command of 3	Use OpenModSim to send a value of 3 to holding register 100. Read holding registers 101 and 102. The expected value of holding register 101 is 3. The expected value of holding register 102 is 6. Attach a screenshot from ModbusPal in the Result column to show that the expected value was received.	
6	Tear Down	Use OpenModSim to close the connection	

		between the laptop and the Arduino. Unplug the ethernet cables from the Arduino and the laptop. Attach a screenshot from ModbusPal in the Result column to show that the connection is closed.	

Arduino MQTT Test

Purpose

This test establishes a framework for MQTT communications on the Arduino Opto which can be used for future integration. Subscriptions and publications are set up to demonstrate the ability to read telemetry data and to send commands. A full cycle of command and response is tested, enabling us to verify that the command was received properly, and a heartbeat is also established to verify that the communication channel remains open.

Description

In this test, the testing laptop runs the MQTT broker. This broker is set up using Eclipse Mosquitto. The laptop will run at IP address 172.16.0.1 and the Arduino will run at IP address 172.16.0.2.

The connection will be verified using the opta/status topic. The Arduino will publish “online” to this topic when it connects. The Arduino will also establish a Last Will protocol so that the broker will publish “offline” to this topic if the Arduino disconnects unexpectedly.

Telemetry and heartbeat functionality are tested by having the Arduino publish to two topics. The first topic is a device id at opta/device-id, which will publish a device ID with a value of 1234 when it detects a connection. The second topic is at opta/heartbeat, and will send a heartbeat counter updating from 1 to 10, incrementing once per second and resetting from 10 to 1 when it finishes.

Commands are tested by having the Arduino subscribe to one topic and publish to two other topics. The command will be sent from the laptop to the Arduino on the topic opta/commands/input. The laptop publishes to this topic and the Arduino subscribes to it. The Arduino reflects that command back to the laptop by publishing on opta/commands/echo. The Arduino will then multiply the command by 2 and publish that value out on the topic opta/commands/result. This will establish that a command can be sent and verified by reflecting it, and that the Arduino is able to take an action based on that command and send a result back.

The test consists of first reading the device ID to confirm that telemetry is being received. Then the heartbeat is read at least twice to confirm that it is updating by verifying that it shows two different values at two different times. Next, a command value of 10 is sent from the laptop. The reflected value will then be read, expecting a value of 10. The doubled value will then be read, expecting a value of 20. Finally, a command value of 3 is sent from the laptop. The reflected value will be read, expecting a value of 3. The doubled value is read, expecting a value of 6.

Procedure

	Step	Description	Result
0	Initial Conditions	The Arduino and the laptop are connected to the same subnet using wifi. The MQTT connection between the Arduino and the laptop has been established. Attach a screenshot from Mosquitto in the Result column to show that the “online” message was published on the opta/status topic.	
1	Read telemetry	Use Mosquitto to read the topic opta/device-id. The expected value is 1234. Attach a screenshot from Mosquitto in the Result column to show that the expected value was received.	
2	Read heartbeat counter	Use Mosquitto to read the topic opta/heartbeat topic. The expected value is between 1 and 10. Attach a screenshot from Mosquitto in the Result column to show that the expected value was received.	
3	Confirm heartbeat counter is incrementing	Use Mosquitto to read the topic opta/heartbeat topic. The expected value is between 1 and 10, and it should be a different value than that read in step 2. Attach a screenshot from Mosquitto in the Result column to show that the expected value was received.	
4	Send command of 10	Use Mosquitto to publish a value of 10 to the topic opta/commands/input. Use Mosquitto to read the topics opta/commands/echo and opta/commands/result. The expected value of topic opta/commands/echo is 10. The expected value of opta/commands/result is 20. Attach a screenshot from Mosquitto in the Result column to show that the expected value was received.	
5	Send command of 3	Use Mosquitto to publish a value of 3 to the topic opta/commands/input. Use Mosquitto to read the topics opta/commands/echo and	

		opta/commands/result. The expected value of topic opta/commands/echo is 3. The expected value of opta/commands/result is 6. Attach a screenshot from Mosquitto in the Result column to show that the expected value was received.	
6	Tear Down	Use Mosquitto to close the connection between the laptop and the Arduino. Unplug the ethernet cables from the Arduino and the laptop. Attach a screenshot from Mosquitto in the Result column to show that the expected value was received.	

Arduino OpenADR Test

Purpose

This test establishes a framework for OpenADR communications on the Arduino Opto which can be used for future integration. The test will register the Arduino as a shedding device, set up a shedding command from the laptop, and then have the Arduino send a report about the response to the shedding command.

Description

In this test, the testing laptop is set up as the Virtual Top Node (VTN) using the python openleadr library, which implements OpenADR using open source code. The laptop will run at IP address 172.16.0.1 and the Arduino will run at IP address 172.16.0.2. The Arduino will be set up to poll the VTN on the test laptop once per second.

The first step of the test is to register the Arduino. It will send a device id as part of the registration process when the Arduino starts up and connects to the network. That id will be 1234.

The next step is for the user to send a shedding event command using the testing laptop. The SIMPLE signal type will be used, which enables the program manager to send different shedding levels.

A light load shedding event will then be initiated. The user publishes a “1” to the payload field of the SIMPLE object of the XML file to trigger a moderate load shedding. The Arduino will receive this command the next time it polls the VTN. The Arduino will then send an XML file back indicating the amount of load it shed. This value will be 3, which is published in the value field of the oadrUpdateReport object.

A moderate load shedding event will then be initiated. The user publishes a “2” to the payload field of the SIMPLE object of the XML file to trigger a high load shedding. The Arduino will receive this command the next time it polls the VTN. The Arduino will then send an XML file back indicating the amount of load it shed. This value will be 10, which is published in the value field of the oadrUpdateReport object.

Procedure

	Step	Description	Result
0	Initial Conditions	The Arduino and the laptop are connected to the same subnet using wifi.	

		The OpenADR connection between the Arduino and the laptop has been established. Attach a screenshot from the command line in the Result column to show that the device ID was received by the VTN during registration.	
1	Send moderate shed command	Use the command line to send a “1” to the openleadr script and initiate a moderate shed command. The openleadr script will receive the Report from the Arduino with an expected value of 3. It will then print this value to the command line. Attach a screenshot from the command line showing the expected value was received.	
2	Send highshed command	Use the command line to send a “2” to the openleadr script and initiate a moderate shed command. The openleadr script will receive the Report from the Arduino with an expected value of 10. It will then print this value to the command line. Attach a screenshot from the command line showing the expected value was received.	
3	Tear Down	Use the command line to send a “0” to the openleadr script and initiate a moderate shed command. The openleadr script will receive the Report from the Arduino with an expected value of 0. It will then print this value to the command line. Attach a screenshot from the command line showing the expected value was received. Unplug the ethernet cables from the laptop and the Arduino.	